SDSC Summer Institute 2026

Parallel Computing Concepts

Igor Sfiligoi

Who cares about parallel computing?

Who cares about parallel computing?

- All modern hardware is parallel
  - Not even your phone does serial compute anymore!
- If you do not use parallel tools, you will be using only a fraction of hardware resources
  - I.e., it will take a long time to finish your compute task

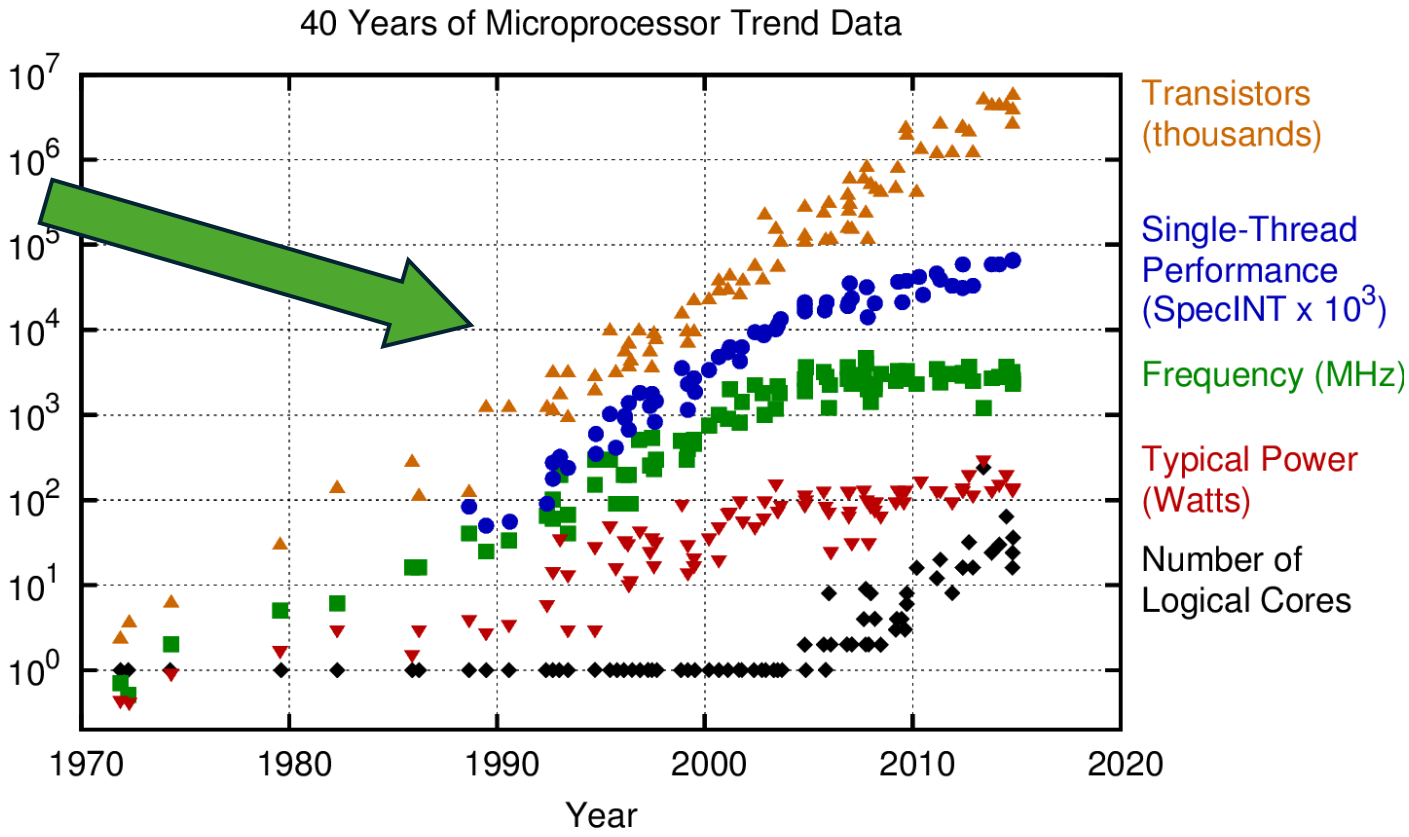
Who cares about parallel computing?

- All modern hardware is parallel
  - Not even your phone does serial compute anymore!
- If you do not use parallel tools, you will be using only a fraction of hardware resources
  - I.e., it will take a long time to finish your compute task
- You likely have access to more than one compute device
  - You can solve a problem faster if you use them all at the same time!
- Some problems are just too big to fit on a single compute node
  - Using multiple nodes may be the only way to solve the problem at all!

Why is all hardware parallel?

Why is all hardware parallel?

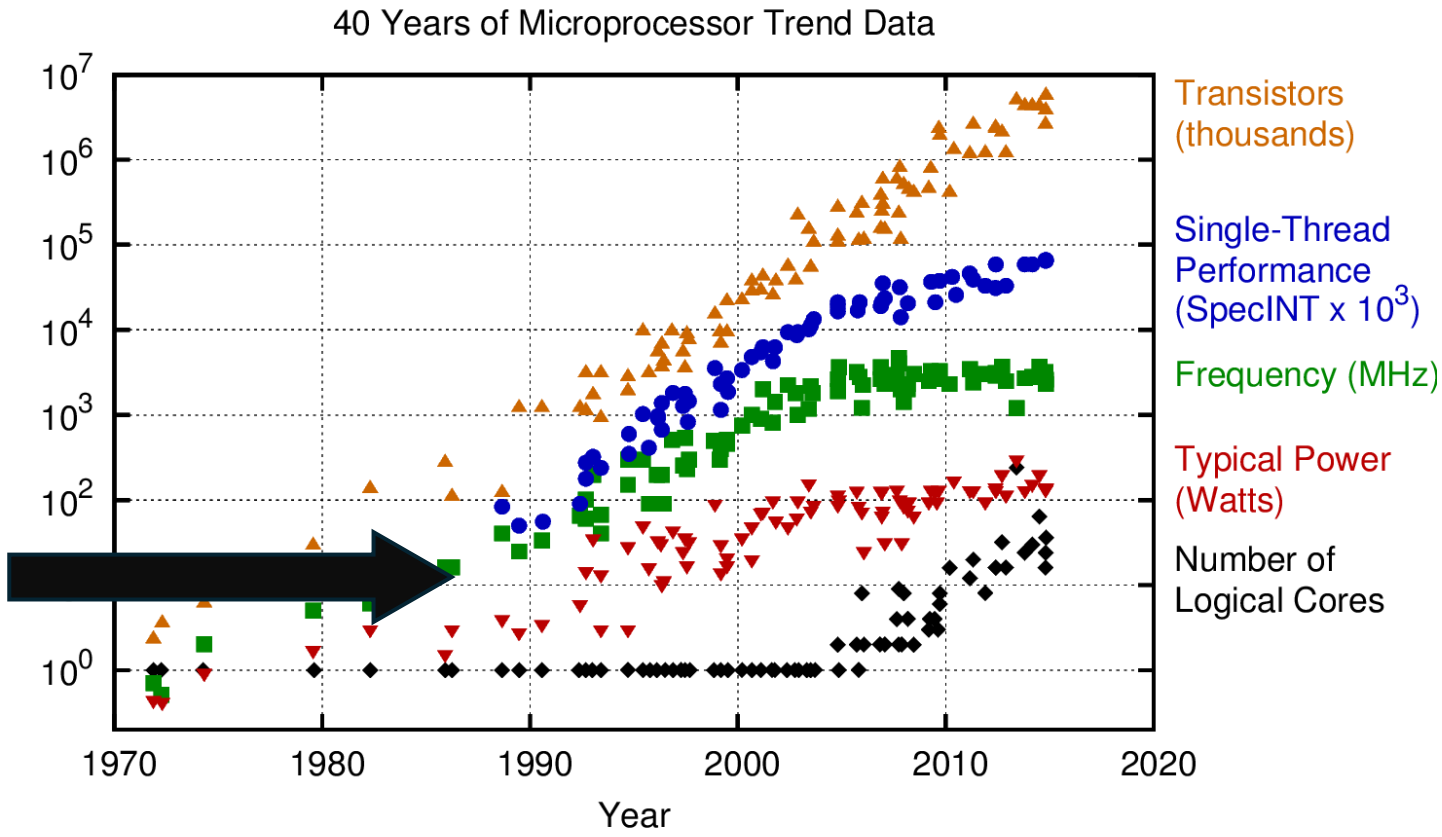
- Scalar, single-core CPU performance stopped scaling long ago
  - We hit fundamental physics limits on how fast we can move to the next instruction



Credits: https://www.karlrupp.net/2015/06/40-years-of-microprocessor-trend-data/

Why is all hardware parallel?

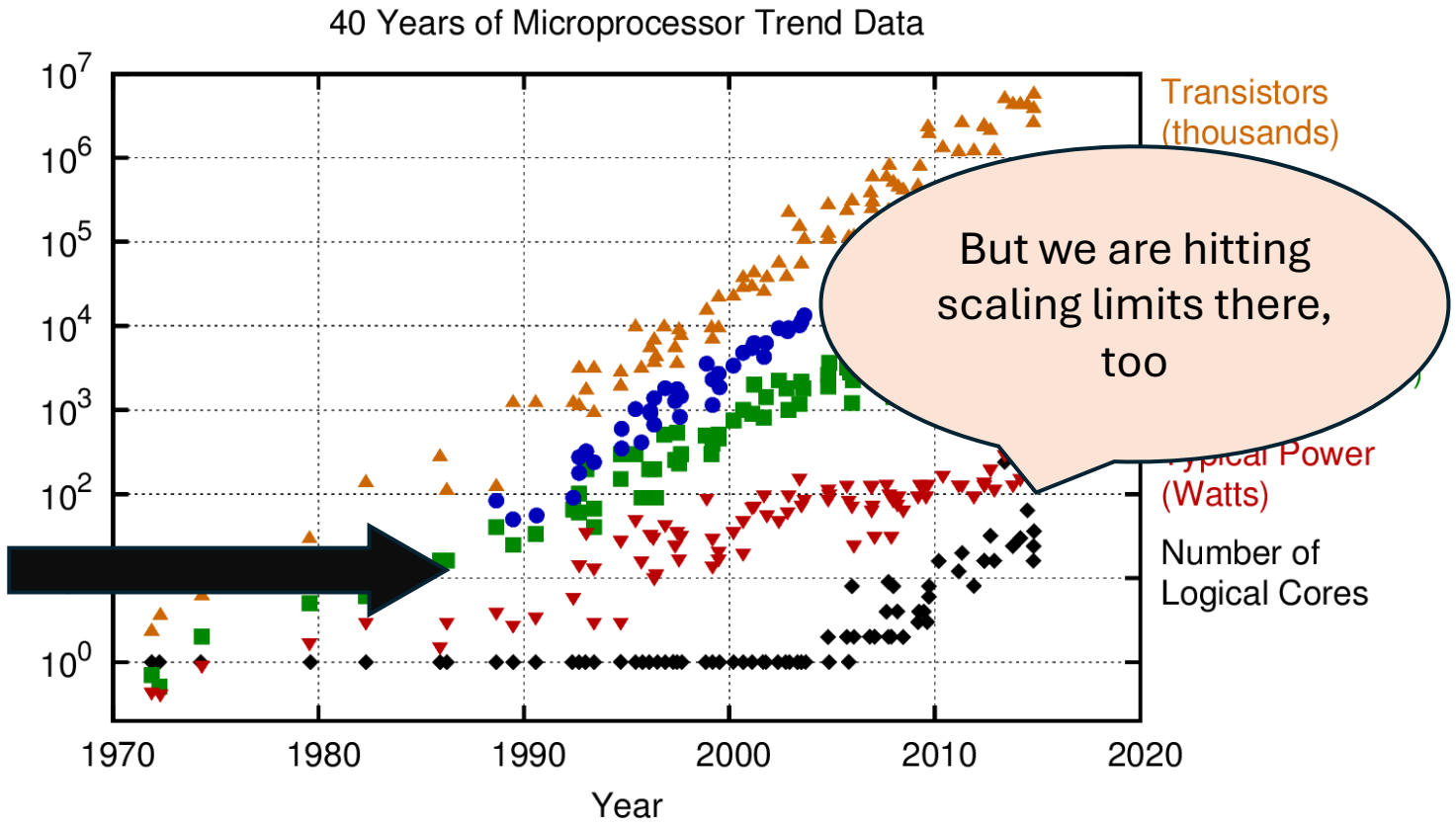
- Scalar, single-core CPU performance stopped scaling long ago
- All modern CPUs have many cores
  - Like having many single-core CPUs in a single package



Credits: https://www.karlrupp.net/2015/06/40-years-of-microprocessor-trend-data/

Why is all hardware parallel?

- Scalar, single-core CPU performance stopped scaling long ago
- All modern CPUs have many cores
  - Like having many single-core CPUs in a single package



Credits: https://www.karlrupp.net/2015/06/40-years-of-microprocessor-trend-data/

Why is all hardware parallel?

- Scalar, single-core CPU performance stopped scaling long ago
- All modern CPUs have many cores, but scaling is slowing
- All modern CPU cores are vectorized
  - Each instruction works on multiple scalar values (a vector)
  - For example, Expanse CPUs use a 256bit vector width, e.g. 8x int, 8x float, 4x double
- Why vector instructions?
  - From a hardware point of view, much easier than adding more scalar cores, together allow to pack more compute into a single CPU chip

Why is all hardware parallel?

- Scalar, single-core CPU performance stopped scaling long ago
- All modern CPUs have many cores, but scaling is slowing
- All modern CPU cores are vectorized
- GPUs are conceptually very similar
  - Many Streaming Multiprocessors (SMs) – Equivalent to CPU cores
  - Each SM operates on a vector – NVIDIA calls them warps
    (It's actually a set of vectors)
  - Modern GPU SMs also have instructions to operate on a matrix (tensor)
    (Heavily used in ML/AI)

How do you approach parallel programming?

Divide the problem

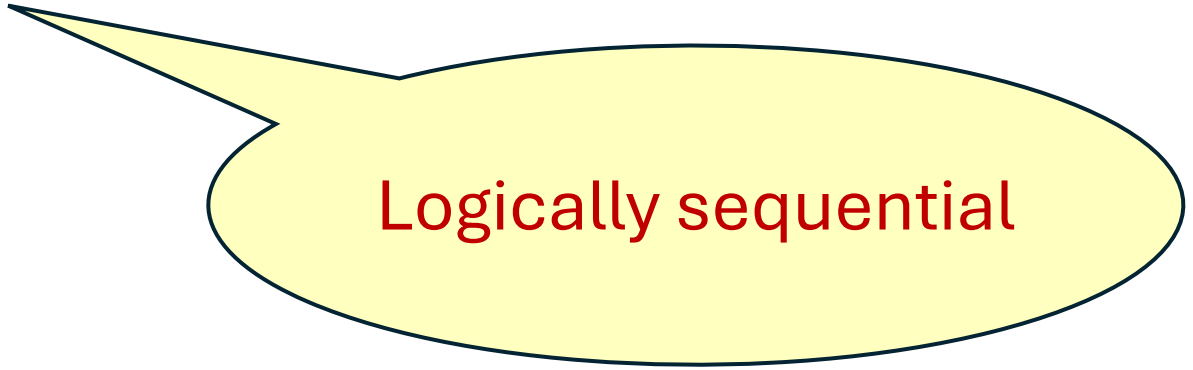
- Using parallel hardware means doing independent operations
  - So, you must be able to divide the problem in many independent pieces
  - If every step requires the result from the previous step, you cannot have parallelism!
- Unfortunately, human brain likes to operate in dependent steps
  - Breaking that mindset can be hard!

Divide the problem

- Using parallel hardware means doing independent operations
  - So, you must be able to divide the problem in many independent pieces
  - If every step requires the result from the previous step, you cannot have parallelism!
- Unfortunately, human brain likes to operate in dependent steps
  - Breaking that mindset can be hard!
- Operations on large datasets are often naturally parallel
  - For example, when operating on large matrices
  - Even if each logical step depends on the previous one, you may be able to parallelize the step itself

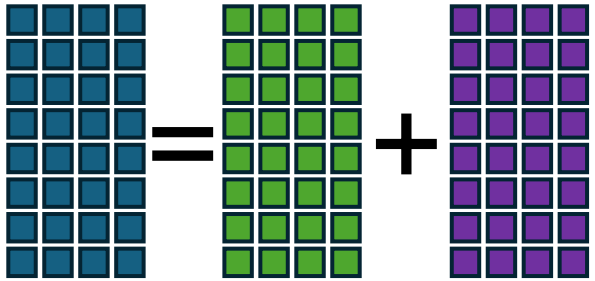
Parallel matrix addition

$$C = A + B$$



Parallel matrix addition

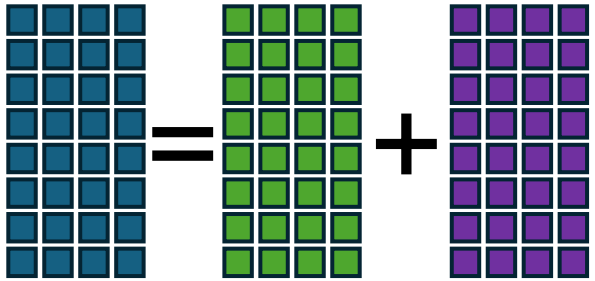
C = A + B



Fully parallel
under the hood

Parallel matrix addition

C = A + B



Fully parallel
under the hood

```
Parallel_for col in range(Ncol):
  Parallel_for row in range(Nrow):
    C[row,col] = A[row,col]  + B[row,col]
```

```
Parallel_for i in range(Ncol*Nrow):
  col  = i / Nrow
  row = i % Nrow
  C[row,col] = A[row,col]  + B[row,col]
```

Problem splitting granularity

- Ideally, you want to be able split the problem only once
  - Perform all the steps independently
  - And collect the pieces at the end
  - Often called **Pleasantly Parallel Computing**
- Unfortunately, not always possible
  - Dependencies between sub-problems are much more common
  - But you should still strive to find sequences of steps where the problem stays pleasantly parallel!

Problem splitting granularity

- Ideally, you want to be able split the problem only once
  - Perform all the steps independently
  - And collect the pieces at the end
  - Often called **Pleasantly Parallel Computing**
- Unfortunately, not always possible
  - Hopefully, at least some sequences of steps are pleasantly parallel
- And you need enough pieces to use all available hardware
  - Virtually no problem can be split forever!
- Some steps may allow for more splitting that others
  - Top-level, all-step splitting may not be the right choice, even when possible!

Why worry about splitting granularity?

Why worry about splitting granularity?

- Each time you have a dependency,
  you must synchronize 2 or more sub-problems
  - Which is expensive!
- Why is synchronization expensive?
  - It is effectively sequential
  - It is very sensitive to jitter
  - It requires data movement

Even a little sequential code adds up

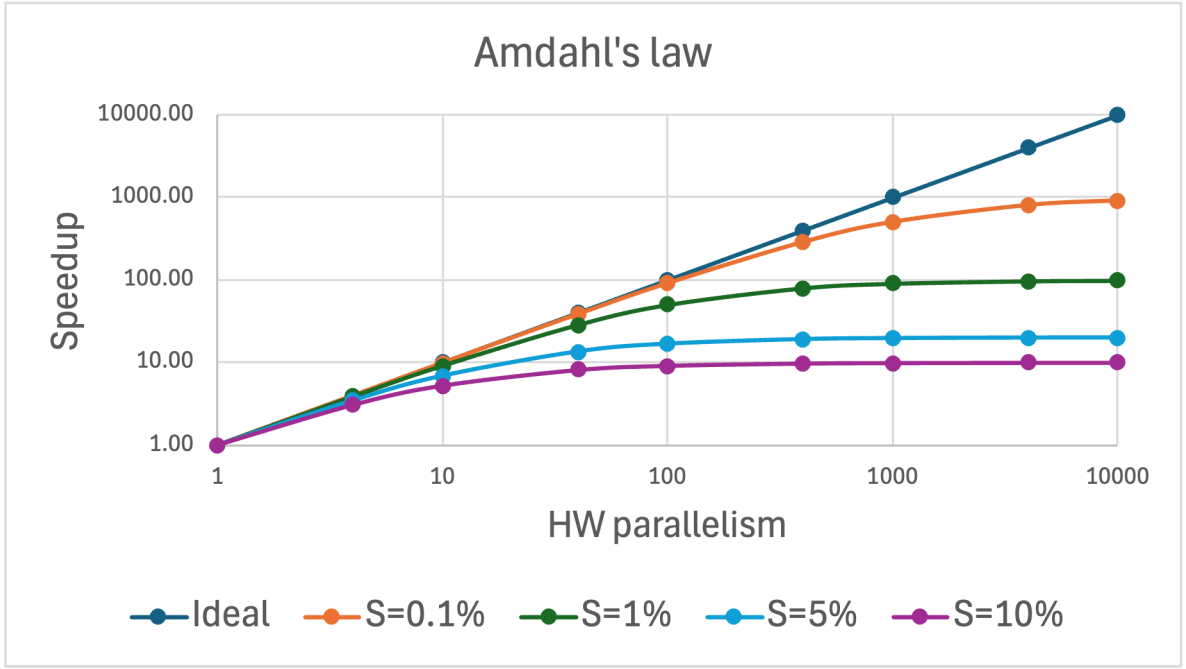
- Every time we are forced to do some sequential operations, we limit how much we can speedup an application
- The theoretical limit is known as **Amdahl's law**
  - P is the fraction of the code that can be parallelized
  - S is the fraction of the code that must be run sequentially (S = 1– P)
  - N = hardware parallelism

$$speedup(N) = \frac{1}{(1-P) + \frac{P}{N}}$$

Even a little sequential code adds up

- Every time we are forced to do some sequential operations, we limit how much we can speedup an application
- The theoretical limit is known as **Amdahl's law**

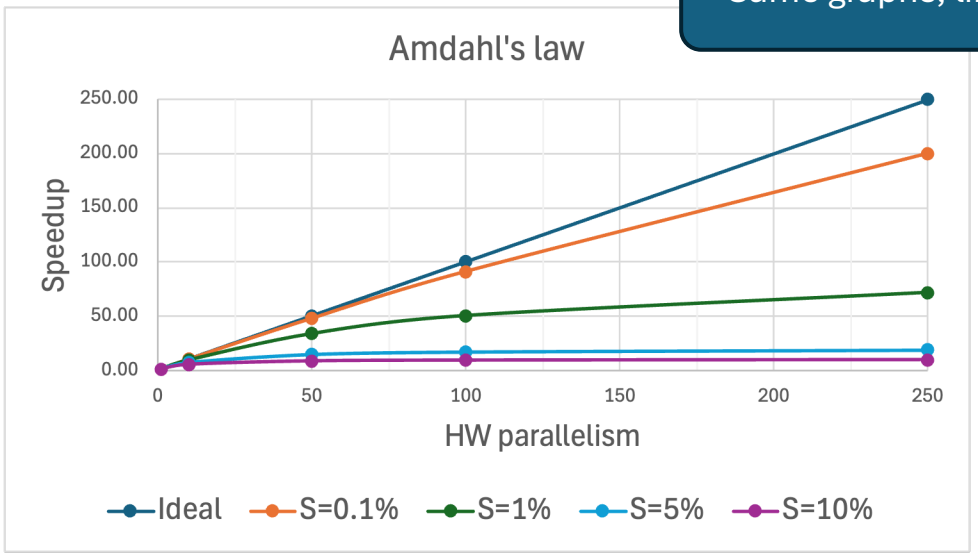
Even with 0.1% sequential code we cannot fully utilize a 1000-way parallel system
(1% too much for a 100-way parallel system)

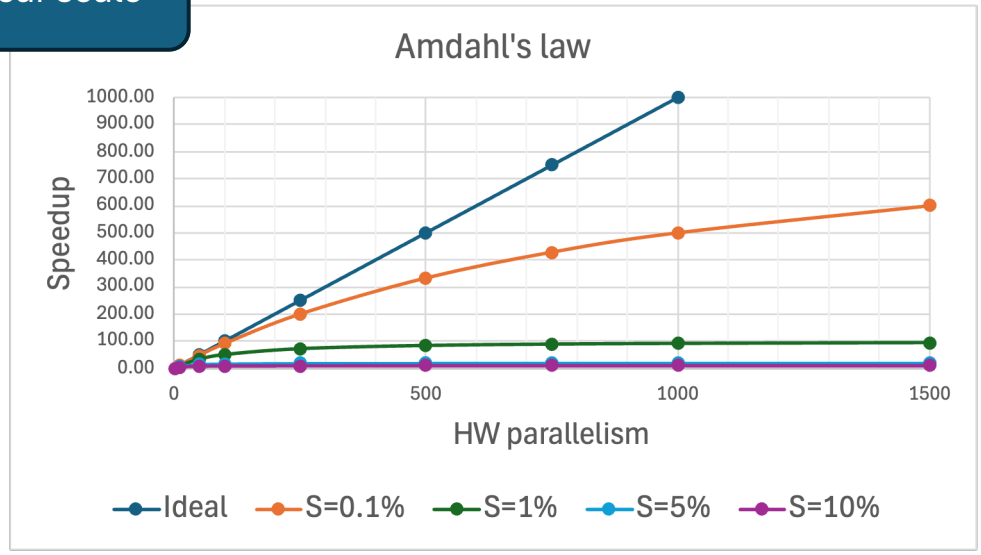


Even a little sequential code adds up

- Every time we are forced to do some sequential operations, we limit how much we can speedup an application
- The theoretical limit is known as **Amdahl's law**

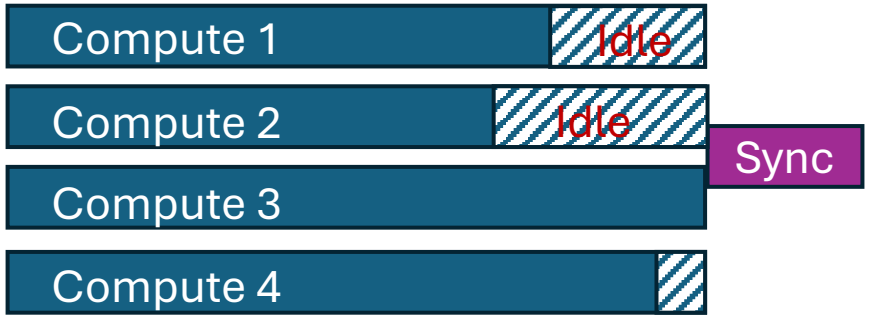
Same graphs, linear scale





Slowest sub-problem dictates overall speed

- Not all sub-problem steps will proceed at the same speed = Jitter
  - HW differences
  - External constraints (e.g. memory access)
  - Different logic paths (e.g. if (i=0) {…} else {…})
- Not a problem as long as each proceeds independently
  - But must wait for the slowest one at synchronization points!



The data movement cost

- Modern processors are incredibly fast
  - At 2Ghz, each core can process one operation every 0.5 ns (nanosecond)
- Getting data in and out of a core is much slower
  - ~10 ns from a nearby core (20 cycles)
  - 100+ ns from a different CPU on the same node (300 cycles)
  - 1000+ ns from a different HPC node (5000 cycles)
- Anything but vector-level synchronization is thus very expensive
  - Especially on multi-node setups

Asynchronous data exchange

Synchronization not always needed

- As we saw, synchronization can be very expensive
- Some dependencies can be solved without full synchronization, for example
  - Counters and flags
  - Pipelines
  - Rare event handling

The multi-core read-write problem

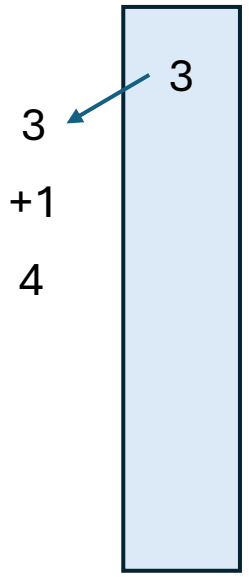
- In most CPU systems,
  many cores can access a memory location of interest
  - a “variable”
- When one core writes to a variable,
  when will the other cores see the updated value?
  - In general, there is no guarantee!
  - Can take 100+ ops before all cores see it

Flags are easy – if not used for decisions

- Let's consider flags
  - Boolean values that can only we switched one way
- If used in isolation, if two cores flip the flag independently, the end value will still be the same
- When the value of the flag is used
  - Another core may take the wrong decision!
  - Only OK if flag was a minor optimization, and the delay only results in useless work

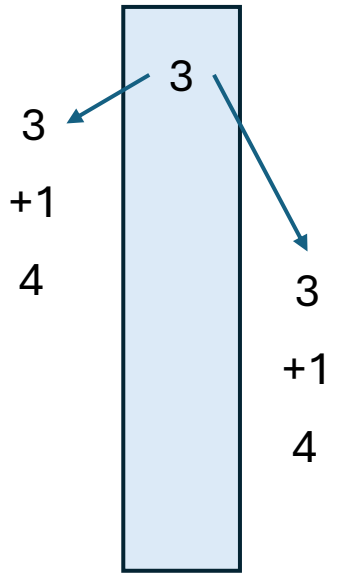
Counters need atomic treatment

- A counter cannot rely on standard memory semantics
  - For example, an increment is really composed of 3 operations
    1. Fetch value of variable from RAM
    2. Add 1 to that value
    3. Store the result in the variable to RAM
  - If two cores try to do the same in parallel, we will end up with just one increment!
  - This is a classical **race condition**



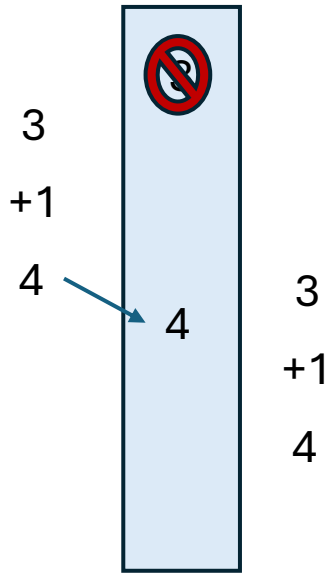
Counters need atomic treatment

- A counter cannot rely on standard memory semantics
  - For example, an increment is really composed of 3 operations
    1. Fetch value of variable from RAM
    2. Add 1 to that value
    3. Store the result in the variable to RAM
  - If two cores try to do the same in parallel, we will end up with just one increment!
  - This is a classical **race condition**



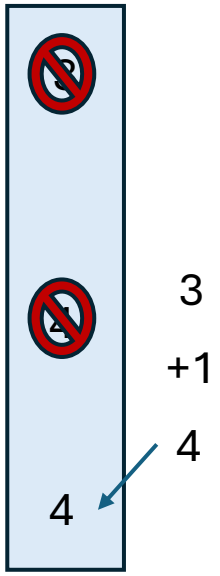
Counters need atomic treatment

- A counter cannot rely on standard memory semantics
  - For example, an increment is really composed of 3 operations
    1. Fetch value of variable from RAM
    2. Add 1 to that value
    3. Store the result in the variable to RAM
  - If two cores try to do the same in parallel, we will end up with just one increment!
  - This is a classical **race condition**



Counters need atomic treatment

- A counter cannot rely on standard memory semantics
  - For example, an increment is really composed of 3 operations
    1. Fetch value of variable from RAM
    2. Add 1 to that value
    3. Store the result in the variable to RAM
  - If two cores try to do the same in parallel, we will end up with just one increment!
  - This is a classical **race condition**



Counters need atomic treatment

- A counter cannot rely on standard memory semantics
  - It leads to a **race condition**
- Most CPUs provide **atomic operations**
  - For the most common operations
  - They guarantee that a value is changed in memory **without race conditions** (You don't have to worry how it is done)
  - Of course, it is substantially more expensive, so use only when needed
- Can be used for flags used for decisions, too

Complex logic needs locks

- Atomics only good for simple operations
- When you must avoid race conditions over multiple variables, use a lock
  (Various names, e.g., mutex, critical section)
  - Since only one core can hold a lock at any time, no race conditions
  - Atomic operations still recommended to avoid timing issues
- If asking for more than one lock, beware **deadlocks**
  - For example
    - Core 1 gets lock1, then waits for lock2
    - Core 2 gets lock2, then waits for lock1

Locks great for rare conditions

- Obtaining a lock can be expensive
  - And potentially serializes the execution of the algorithm
- Only use sparingly
  - But a great solution for dealing with rare conditions!

Data partitioning

Data-Parallel Computing

- While you can parallelize certain algorithms, **most commonly parallelism comes from operating on vast amounts of data**
- This brings the question:
  How do you partition the data?
- Can you partition once, and keep it across all steps?
  Or will each step need a different partitioning?

Shared vs partitioned RAM

- Most modern CPUs allow all cores to access the same RAM
  - Thus, allowing both fixed and dynamic partitioning of problems
  - Same true for multiple CPUs inside a single node
- But when moving compute to **multiple HPC nodes**, the RAM becomes partitioned
  - Changing problem partitioning requires
    - Duplication of read-only data
    - Moving large amounts of data between nodes
  - Dynamic problem partitioning thus very expensive!

Shared vs partitioned RAM

- Most modern CPUs allow all cores to access the same RAM
  - Thus, allowing both fixed and dynamic partitioning of problems
  - Same true for multiple CPUs inside a single node
- But when moving compute to **multip** the RAM becomes partitioned
  - Changing problem partitioning requir
    - Duplication of read-only data
    - Moving large amounts of data between nodes
  - Dynamic problem partitioning thus very expensive!

Even inside a single node, dynamic partitioning comes with noticeable cost.

Memory hierarchies

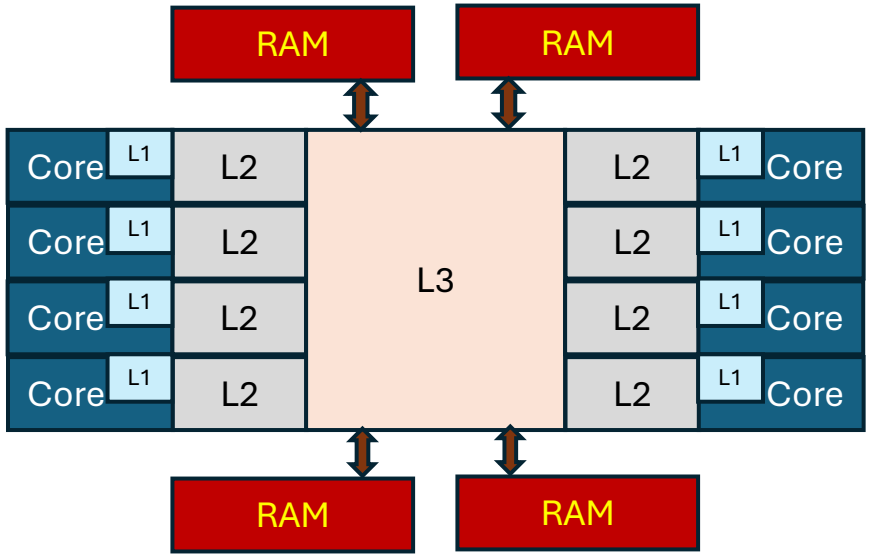
Memory access is slow

- Fetching a value from RAM is very slow
  - 100+ ns
  - Compare that to 0.5 ns for an arithmetic operation!
- Processors operate on internal memory, called **registers**
  - Which are just as fast as the compute part
  - However, most CPUs limited to O(10) registers per thread
  - Which is **too small for most problems**!
  - **Virtually all data thus resident in system memory**, loaded into registers as-needed (and stored back, too)

Memory access is slow

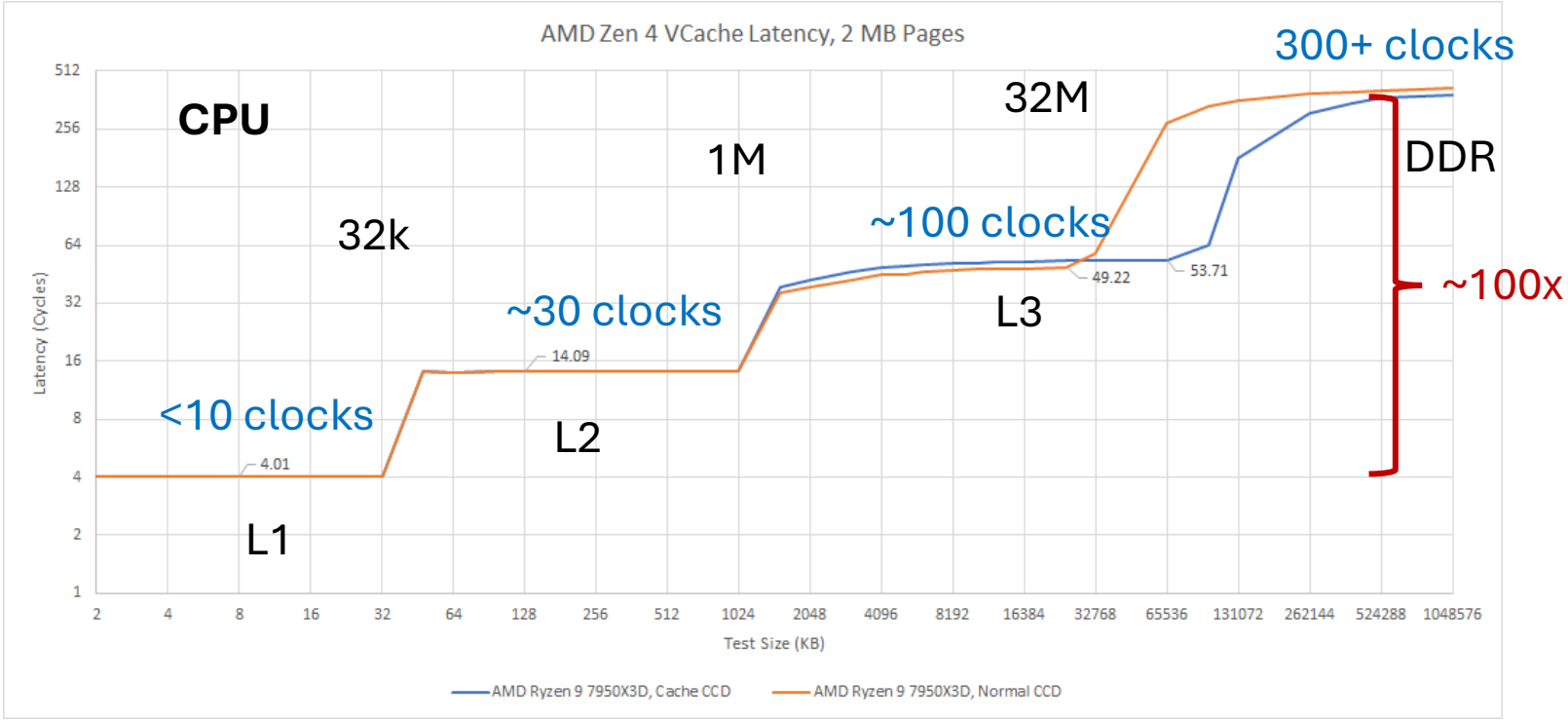
- Fetching a value from RAM is very slow
  - 100+ ns
  - Compare that to 0.5 ns for an arithmetic operation!
- Most CPUs and GPUs have on-chip caches to compensate
  - If a ”variable” is reused soon enough, it will be found in the cache
  - Several levels
    - L1 - fast but small
    - L2+ - significantly larger, but slower

Example CPU memory hierarchy



A typical CPU memory hierarchy

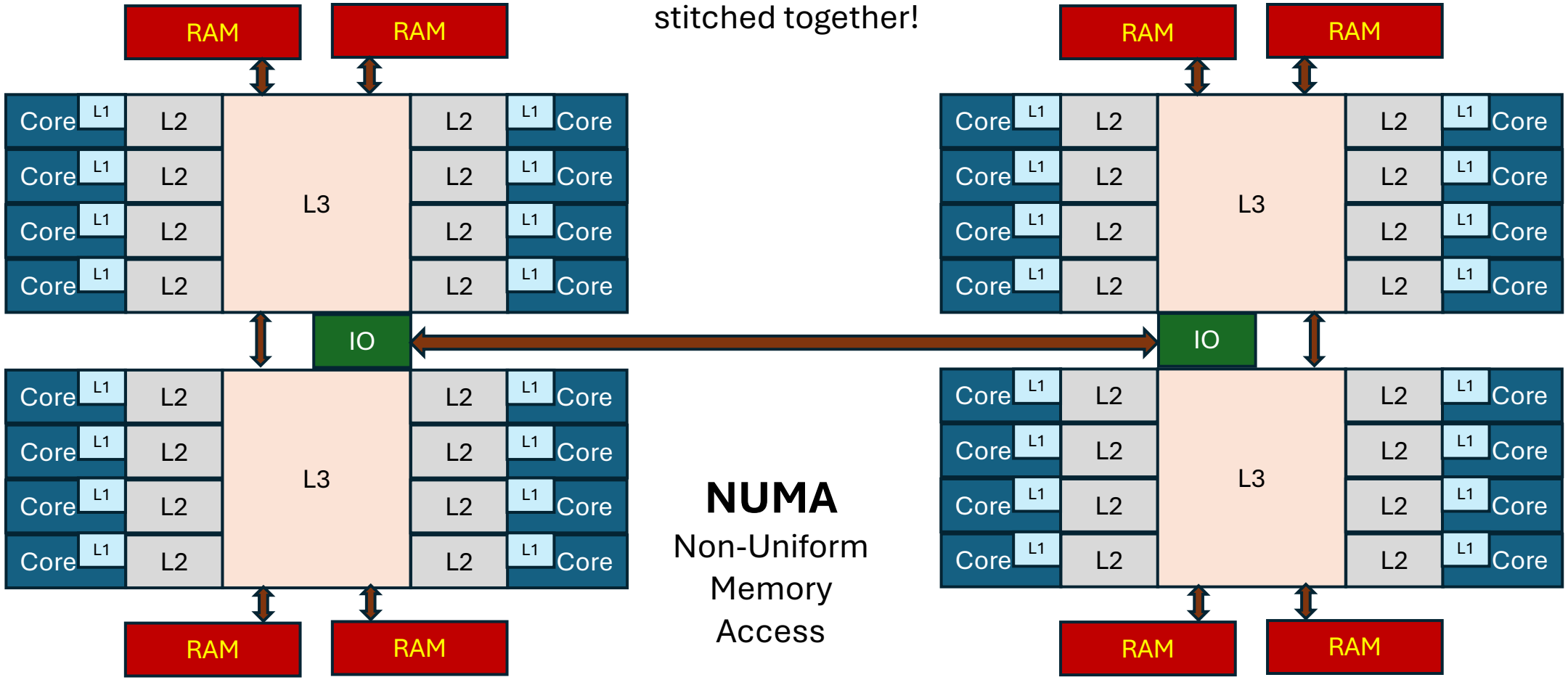
- Here is an example of a modern CPU
- Exact cache size and latency will vary by CPU model
  - But not by very much



Credits https://chipsandcheese.com/2023/07/17/genoa-x-server-v-cache-round-2/

Most HPC nodes even more complex

It is really many building blocks stitched together!



NUMA
Non-Uniform Memory Access

Implications on data partitioning

- Even is RAM looks shared between all cores
  - The caches are not!
- The application does not explicitly see the caches
  - But the underlying hardware implicitly does
- Try to use fixed problem partitioning,
  so each core can make ideal use of its caches
  - With dynamic problem partitioning,
    data must be moved between caches or fetched from RAM
    (even though the application may not be aware of it)

Implications on parallel speedup

- RAM bandwidth is never sized for the worst case scenario of all cores accessing it at the same time
  - RAM can only concurrently serve a few cores at a time!
  - If more cores try to access it, the requests are serialized

- You can only linearly scale performance, if you can fit in L2
  - Anything beyond that will result in **memory contention**!

In summary

- All modern compute hardware is parallel
  - So parallel computing is unavoidable
- The easiest way to use parallel hardware is Data-Parallel Computing
  - But make sure you avoid race conditions
- Memory access speed often the limiting factor
  - Think carefully how you split the problem
- Using multiple nodes requires additional thought